

CATÁLOGO AGROFLORESTAL CEDVB

Documento de Escopo e Arquitetura

Versão 1.0

Baseado no Modelo de Visões 4+1 (KRUCHTEN, 1995)

Informações do Projeto

Projeto	Catálogo Agroflorestal CEDVB
Instituição atendida	Escola CEDVB — Distrito Federal
Desenvolvedor responsável	Rafael Araujo Queiroz
Repositório	https://github.com/RafaelQueiroz7/Catalogo-Agrofloresta
Ambiente de produção	https://catalogo-agrofloresta.vercel.app

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
29/08/2026	1.0	Criação do documento de arquitetura	Rafael Araujo Queiroz

Sumário

1 Introdução

1.1 Propósito

1.2 Escopo

2 Representação Arquitetural

2.1 Definições

2.2 Justificativa da Escolha

2.3 Detalhamento

2.4 Metas e Restrições Arquiteturais

2.5 Visões

2.5.1 Visão de Uso

2.5.2 Visão de Organização Lógica

2.5.3 Visão Estrutural

2.6 Visão de Implantação

2.7 Restrições Adicionais

3 Bibliografia

Nota: os títulos acima usam estilos de título nativos do Word (Heading 1/2/3). Para gerar um sumário automático com numeração de página, use Referências → Sumário no Word.

1. Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema Catálogo Agroflorestal CEDVB, desenvolvido para o registro, a organização e a divulgação das espécies vegetais utilizadas no sistema agroflorestal da escola. O objetivo deste documento é fornecer uma visão abrangente da arquitetura de software para desenvolvedores, mantenedores e demais interessados nos aspectos técnicos e tecnológicos envolvidos na construção e evolução do sistema.

1.2 Escopo

O escopo do produto compreende o desenvolvimento de uma plataforma web pública de catálogo e guia de espécies agroflorestais, permitindo que qualquer visitante consulte informações detalhadas sobre cada planta cultivada — características, origem, forma de cultivo, propriedades, usos e cuidados. A plataforma conta ainda com uma área administrativa protegida por senha, na qual a equipe responsável pela agrofloresta pode cadastrar novas espécies, incluindo o envio de material visual produzido pela comunidade escolar (fotografias reais, aquarelas, carimbos botânicos e mapas de origem, em formato de imagem ou PDF).

2. Representação Arquitetural

2.1 Definições

O sistema Catálogo Agroflorestal CEDVB segue uma arquitetura full-stack baseada em React Server Components, viabilizada pelo App Router do framework Next.js. Diferentemente de uma arquitetura cliente-servidor tradicional, com uma API REST separada entre front-end e back-end, o Next.js permite que os próprios componentes de apresentação (Server Components) consultem o banco de dados diretamente durante a renderização no servidor, e que operações de escrita (como o cadastro de uma espécie ou o login administrativo) sejam realizadas por meio de Server Actions — funções que executam exclusivamente no servidor e são invocadas diretamente por formulários no cliente, sem a necessidade de se construir e manter manualmente endpoints REST/JSON.

Essa abordagem pode ser organizada, de forma conceitualmente próxima ao padrão MVC, em três responsabilidades principais:

- Apresentação (View): componentes React responsáveis por renderizar a interface, tanto no servidor (Server Components, como as páginas do catálogo e do guia de espécie) quanto no cliente (Client Components, como os formulários interativos de cadastro e login);
- Controle (Server Actions e Middleware): funções que executam no servidor, recebem as submissões dos formulários, validam os dados de entrada, processam o envio de arquivos e aplicam as regras de acesso à área administrativa;
- Dados (Model): o Prisma ORM, atuando como camada de abstração entre o código da aplicação e o banco de dados PostgreSQL, complementado por um driver adapter responsável por estabelecer a conexão real com o banco.

2.2 Justificativa da Escolha

A escolha por essa arquitetura considerou o escopo do projeto, o tamanho da equipe de desenvolvimento (um único desenvolvedor responsável pela manutenção) e as características do produto descritas no propósito deste documento.

O uso do Next.js com Server Components e Server Actions elimina a necessidade de projetar, implementar e manter uma API REST independente, reduzindo a superfície de código e a quantidade de camadas a serem sincronizadas a cada alteração — um fator especialmente relevante para um projeto mantido por um desenvolvedor único. Além disso, a renderização no servidor (SSR) beneficia diretamente o propósito do catálogo: como o conteúdo é predominantemente informativo e destinado à consulta pública, páginas renderizadas no servidor carregam mais rapidamente e são mais bem indexadas por mecanismos de busca do que uma aplicação puramente client-side.

A adoção do PostgreSQL, por meio do Prisma ORM, foi motivada pela natureza bem definida e relacional dos dados do domínio (uma entidade central de Espécie, com campos estruturados e tipados). O uso do Supabase como provedor único de banco de dados e de armazenamento de arquivos (Storage) reduz a quantidade de serviços externos a serem geridos, o que é vantajoso para um projeto de porte pequeno e manutenção solo. Por fim, a hospedagem na Vercel foi escolhida por sua integração nativa com o Next.js, pelo modelo de deploy contínuo a partir do repositório Git e pela disponibilidade de um plano gratuito compatível com o orçamento do projeto.

Uma arquitetura de microsserviços foi descartada por aumentar significativamente a complexidade de desenvolvimento, implantação e comunicação entre módulos, sem trazer benefícios proporcionais ao escopo atual do sistema — que possui uma única entidade de domínio e um único perfil administrativo.

2.3 Detalhamento

A estrutura do sistema é organizada em camadas de responsabilidade dentro do próprio projeto Next.js, conforme ilustra a representação a seguir:

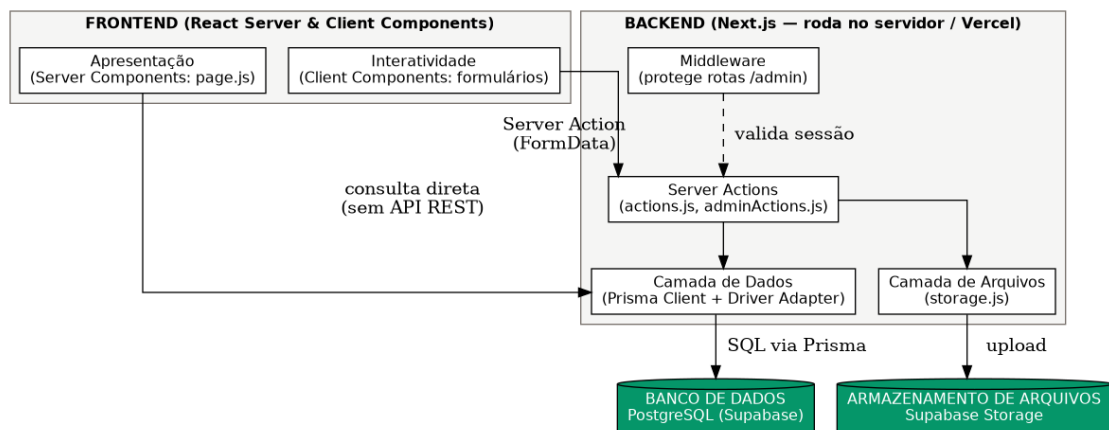


Figura 1 – Estrutura arquitetural do Catálogo Agroflorestal CEDVB (fonte: autoria própria, 2026)

Frontend — Componentes de Servidor e Cliente

Implementado com React e estilizado com Tailwind CSS, é responsável por renderizar as interfaces com as quais o usuário interage. As páginas do catálogo e do guia de espécie são Server Components, que buscam os dados diretamente via Prisma durante a renderização. Já os formulários de cadastro e login são Client Components, responsáveis pela interatividade, pelo estado local de erro e pela submissão dos dados às Server Actions correspondentes.

Backend — Server Actions e Middleware

Responsável por processar as requisições de escrita, aplicar as regras de negócio (como a geração automática de slug a partir do nome popular) e controlar o acesso à área administrativa. No sistema, existem Server Actions específicas para cada operação: cadastrarEspécie, entrarAdmin e sairAdmin. Um middleware intercepta toda requisição destinada a rotas administrativas, validando a sessão do usuário antes de permitir o acesso.

Banco de Dados — PostgreSQL (Supabase)

Camada de persistência responsável por armazenar de forma íntegra todos os registros de espécies cadastradas. A comunicação com essa camada é feita exclusivamente pelo Prisma Client, por meio de um driver adapter, garantindo que nenhuma outra camada acesse diretamente o banco de dados.

Armazenamento de Arquivos — Supabase Storage

Camada responsável por armazenar os arquivos de imagem e PDF enviados no cadastro de cada espécie (fotografia real, carimbo botânico, aquarela e mapa de origem). O banco de dados armazena apenas a URL pública de cada arquivo, nunca o arquivo em si.

Instanciação dos Elementos Arquiteturais

A tabela a seguir relaciona os módulos concretos do código-fonte às suas responsabilidades arquiteturas:

- actions.js — Server Action cadastrarEspecie: valida os campos obrigatórios, gera o slug, envia os arquivos ao Storage e persiste o registro via Prisma;
- adminActions.js — Server Actions entrarAdmin e sairAdmin: validam a senha administrativa e gerenciam o cookie de sessão;
- middleware.js — intercepta requisições às rotas /admin/*, valida o cookie de sessão e redireciona usuários não autenticados;
- server/db.js — instancia o Prisma Client com o driver adapter, expondo o acesso ao banco de dados ao restante da aplicação;
- server/storage.js — encapsula o envio de arquivos ao Supabase Storage e a obtenção da URL pública;
- lib/auth.js — gera o token de sessão a partir da senha administrativa e de um segredo do servidor;
- lib/utlis.js — funções utilitárias, como a identificação de arquivos em formato PDF para exibição condicional.

O fluxo de cadastro de uma nova espécie, que concentra a principal regra de negócio do sistema, está detalhado na Figura 3.

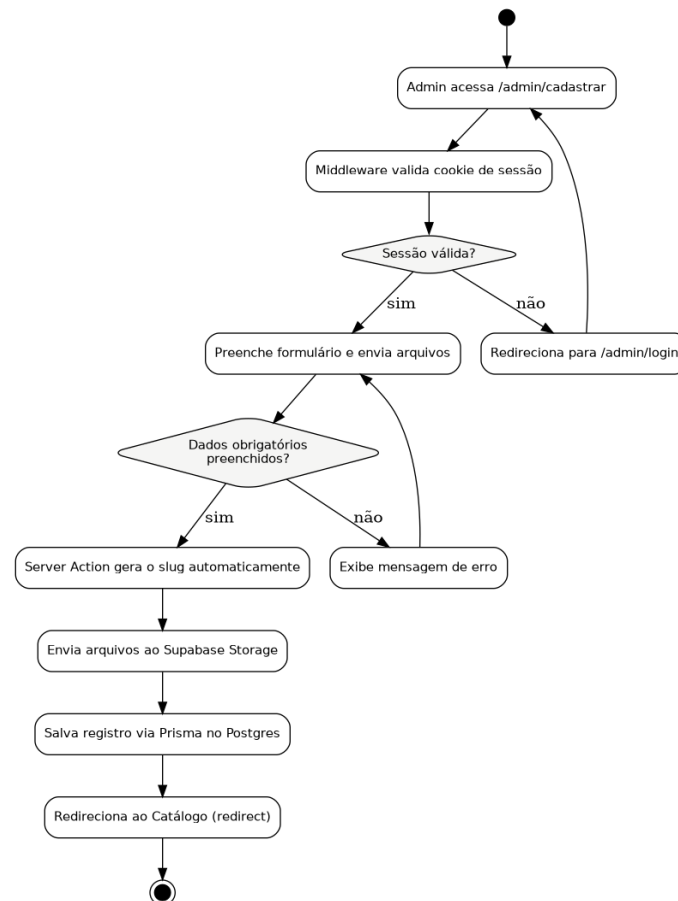


Figura 3 – Diagrama de Atividades: Cadastro de Espécie (fonte: autoria própria, 2026)

2.4 Metas e Restrições Arquiteturais

Metas Arquiteturais:

- O sistema deve apresentar tempo de resposta rápido nas operações principais, como consulta ao catálogo e visualização do guia de cada espécie;
- A plataforma deve possuir interface responsiva, com acesso adequado tanto em computadores quanto em dispositivos móveis;

- O sistema deve ser de fácil uso para pessoas com diferentes níveis de familiaridade com tecnologia, incluindo professores e funcionários da escola;
- O software deve possuir baixo custo operacional, viabilizado pelo uso das camadas gratuitas do Supabase e da Vercel;
- A arquitetura deve ser suficientemente simples para ser mantida por um único desenvolvedor responsável.

Restrições Arquiteturais:

- O desenvolvimento será realizado utilizando JavaScript (React/Next.js) no front-end e back-end, com Tailwind CSS para estilização;
- O banco de dados adotado será o PostgreSQL, hospedado no Supabase;
- O armazenamento de arquivos será realizado no Supabase Storage;
- A hospedagem da aplicação será feita na Vercel, com deploy contínuo a partir da branch de produção do repositório Git;
- O sistema deverá funcionar diretamente em navegadores web modernos, sem necessidade de instalação adicional.

Padrões e diretrizes:

- Nomenclatura de campos do domínio em português (nomePopular, características, localOrigem, entre outros);
- Organização do código em camadas dentro de src/ (app, components, server, lib), separando apresentação, controle e acesso a dados;
- Toda operação de escrita no banco de dados é realizada exclusivamente por meio de Server Actions;
- Uso do GitHub para controle de versão, com deploy automático a partir da branch dev.

2.5 Visões

2.5.1 Visão de Uso

O Catálogo Agroflorestal CEDVB é uma aplicação web responsiva voltada à consulta pública de espécies vegetais e ao cadastro administrativo dessas espécies. Seu objetivo central é permitir que qualquer visitante conheça, em detalhe, as plantas cultivadas na agrofloresta escolar, e que a equipe responsável mantenha esse catálogo atualizado por meio de uma área de acesso restrito.

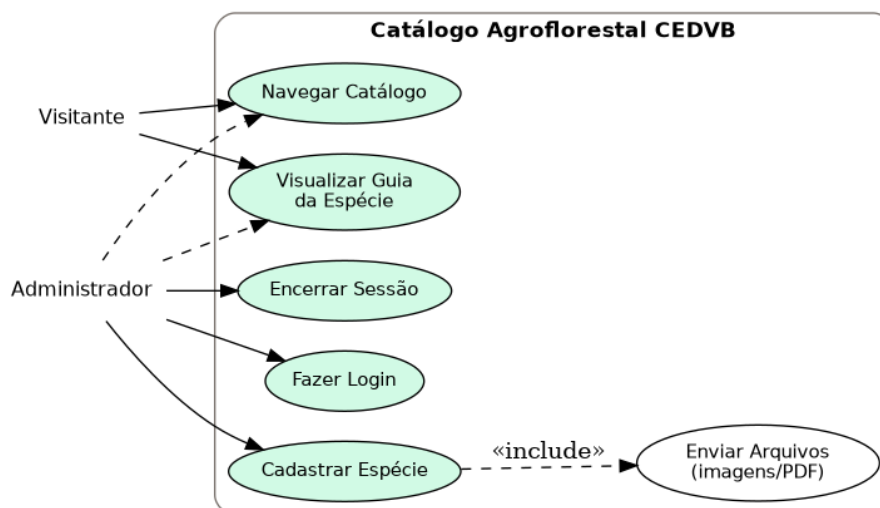


Figura 2 – Diagrama de Casos de Uso do sistema (fonte: autoria própria, 2026)

As interações essenciais do sistema envolvem dois atores: o Visitante, que navega livremente pelo catálogo e pelas páginas-guia de cada espécie, sem necessidade de autenticação; e o Administrador, que herda as ações do

Visitante e, adicionalmente, realiza login, cadastra novas espécies (incluindo o envio de arquivos) e encerra sua sessão.

2.5.2 Visão de Organização Lógica

Partindo do princípio de separação de responsabilidades, o código-fonte do sistema é dividido em pacotes que refletem as camadas de apresentação, controle e dados. O pacote `app/` concentra as rotas e páginas do sistema, correspondendo à camada de apresentação. O pacote das `Server Actions` e do `middleware` concentra a lógica de controle, recebendo as solicitações vindas dos formulários e coordenando as operações necessárias. Por fim, os pacotes `server/` e `lib/` concentram o acesso a dados e os serviços técnicos de apoio (conexão com o banco, upload de arquivos e geração do token de sessão).

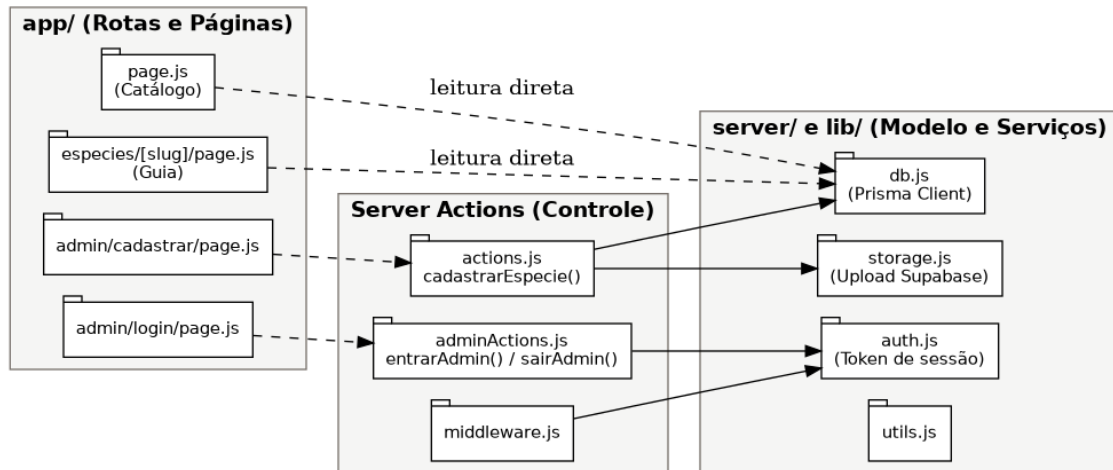


Figura 4 – Diagrama de Pacotes do sistema (fonte: autoria própria, 2026)

As dependências entre os módulos seguem uma direção única — da apresentação para o controle, e do controle para os dados — reduzindo o acoplamento entre as camadas e facilitando a manutenção isolada de cada uma delas.

2.5.3 Visão Estrutural

A visão estrutural descreve a organização física e lógica dos componentes que compõem a aplicação, detalhando como esses elementos se conectam e quais são suas responsabilidades específicas.

O sistema é estruturado em três grandes blocos funcionais:

- **Frontend (Camada de Apresentação):** responsável por renderizar a interface do usuário, capturar entradas de dados e validar formulários no lado do cliente antes do envio;
- **Backend (Camada de Controle e Regras de Negócio):** responsável por processar as `Server Actions`, aplicar validações, gerar o slug de cada espécie, gerenciar a sessão administrativa e coordenar o envio de arquivos ao `Storage`;
- **Persistência (Banco de Dados e Armazenamento):** responsável por armazenar de forma íntegra os registros de espécies (`PostgreSQL`) e os arquivos de imagem/PDF associados a cada uma (`Supabase Storage`).

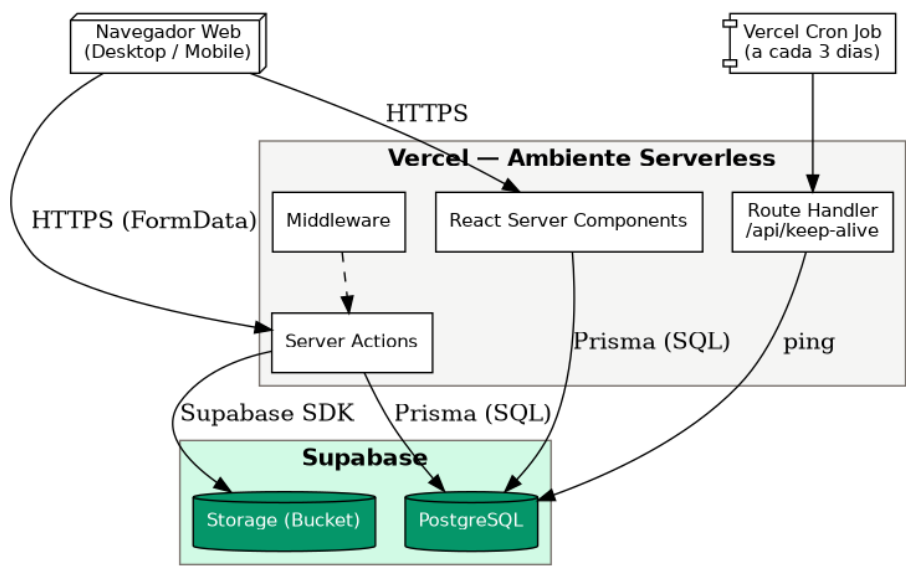


Figura 5 – Diagrama de Componentes (fonte: autoria própria, 2026)

O modelo de dados do sistema foi desenhado em torno de uma única entidade central, *Espécie*, refletindo o escopo enxuto do produto — não há, no momento, outras entidades relacionadas nem múltiplos perfis de usuário armazenados em banco, já que o acesso administrativo é controlado por uma senha única definida em variável de ambiente.

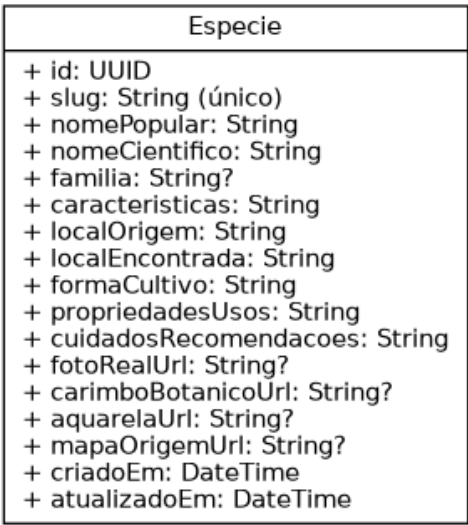


Figura 6 – Diagrama de Classes / Modelo de Dados (fonte: autoria própria, 2026)

A tabela a seguir detalha cada campo da entidade *Espécie*:

Campo	Tipo	Observação
id	UUID	Identificador único, gerado automaticamente
slug	String (único)	Gerado automaticamente a partir do nome popular
nomePopular	String	Obrigatório
nomeCientifico	String	Obrigatório
familia	String (opcional)	Família botânica
caracteristicas	String	Descrição visual da planta

localOrigem	String	Continente ou país de origem
localEncontrada	String	Onde é encontrada no Brasil
formaCultivo	String	Luminosidade, solo, posição na agrofloresta
propriedadesUsos	String	Usos alimentares e/ou medicinais
cuidadosRecomendacoes	String	Espinhos, toxinas, alergias, etc.
fotoRealUrl / carimboBotanicoUrl / aquarelaUrl / mapaOrigemUrl	String (opcional)	URLs dos arquivos no Supabase Storage (imagem ou PDF)
criadoEm / atualizadoEm	DateTime	Preenchidos automaticamente pelo Prisma

Conectores e Protocolos

- Navegador ↔ Servidor Next.js: comunicação via HTTPS, tanto para a renderização das páginas (Server Components) quanto para a submissão de formulários (Server Actions);
- Servidor Next.js ↔ Banco de Dados: conexão realizada por meio do Prisma Client e de um driver adapter específico para PostgreSQL, utilizando consultas SQL para a manipulação dos dados;
- Servidor Next.js ↔ Storage: comunicação por meio do SDK do Supabase, responsável pelo upload dos arquivos e pela obtenção de suas URLs públicas.

2.6 Visão de Implantação

A visão de implantação descreve a topologia física do sistema, ilustrando como o software é distribuído entre os nós de infraestrutura e como esses nós se comunicam. O Catálogo Agroflorestal CEDVB utiliza uma arquitetura baseada em nuvem, garantindo acessibilidade via web sem necessidade de instalação local.

Infraestrutura e Ambiente de Software:

- Nó do Cliente (User Device): qualquer dispositivo, desktop ou móvel, capaz de executar um navegador web moderno;
- Servidor de Aplicação (Vercel): hospeda o Next.js em ambiente serverless, processando tanto a renderização das páginas quanto as Server Actions e o middleware de autenticação;
- Servidor de Banco de Dados (Supabase): nó dedicado à execução do PostgreSQL, onde os registros das espécies são armazenados;
- Servidor de Armazenamento (Supabase Storage): nó dedicado ao armazenamento dos arquivos de imagem e PDF enviados no cadastro.

Como medida específica desta implantação, um Cron Job da Vercel realiza uma consulta trivial ao banco de dados a cada três dias, por meio de uma rota dedicada (/api/keep-alive). Essa automação evita que o projeto gratuito do Supabase seja pausado por inatividade, garantindo a disponibilidade contínua do sistema mesmo em períodos sem acesso de usuários reais.

Justificativa da Infraestrutura:

A escolha por uma implantação em nuvem (Vercel e Supabase) se justifica pela meta de disponibilidade sem custos de infraestrutura própria, adequada ao orçamento e à escala de um projeto escolar. A separação física entre o servidor de aplicação e o banco de dados reduz o acoplamento e facilita manutenções isoladas, atendendo aos requisitos de manutenibilidade definidos para o projeto.

2.7 Restrições Adicionais

O sistema será acessado diretamente pela internet por meio de navegadores web. A consulta ao catálogo é pública e não requer autenticação; já o cadastro de novas espécies exige autenticação por senha única para acesso à área administrativa, garantindo que apenas pessoas autorizadas pela escola possam alterar o conteúdo do catálogo.

Características de Qualidade:

- Usabilidade: interface intuitiva, simples e responsiva, permitindo fácil utilização por visitantes com diferentes níveis de familiaridade com tecnologia;
- Confiabilidade: os registros cadastrados permanecem armazenados corretamente no banco de dados, com validação de campos obrigatórios antes da gravação;
- Portabilidade: o sistema funciona em diferentes dispositivos e navegadores modernos, sem necessidade de instalação;
- Desempenho: as páginas de consulta pública são otimizadas por meio de renderização no servidor, favorecendo tempos de carregamento reduzidos;
- Segurança: a área administrativa é protegida por senha e por um cookie de sessão assinado, validado a cada requisição pelo middleware; variáveis sensíveis (chaves de acesso ao banco e ao Storage) nunca são expostas ao navegador.

Os perfis de acesso previstos são:

- Visitante: pode navegar pelo catálogo e visualizar o guia completo de qualquer espécie cadastrada, sem necessidade de autenticação;
- Administrador: perfil único, autenticado por senha compartilhada, responsável pelo cadastro de novas espécies e pela manutenção do conteúdo do catálogo.

Essas restrições e características foram definidas com o objetivo de garantir segurança, acessibilidade e estabilidade no funcionamento do Catálogo Agroflorestal CEDVB, contribuindo para a facilidade de uso da plataforma pela comunidade escolar e para a manutenção adequada do sistema ao longo de sua evolução.

3. Bibliografia

- KRUCHTEN, Philippe. The 4+1 View Model of Architecture. IEEE Software, v. 12, n. 6, p. 42–50, nov. 1995.
- NEXT.JS. Documentation. Disponível em: <https://nextjs.org/docs>. Acesso em: 29 ago. 2026.
- PRISMA. Documentation. Disponível em: <https://www.prisma.io/docs>. Acesso em: 29 ago. 2026.
- SUPABASE. Documentation. Disponível em: <https://supabase.com/docs>. Acesso em: 29 ago. 2026.
- REACT. Documentation. Disponível em: <https://react.dev>. Acesso em: 29 ago. 2026.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education, 2019.